WritUploaded

AI-Powered Judiciary Drafting Platform

System Architecture Document

Phase 1 + Phase 2 — Complete Technical Blueprint

Document	WritUploaded Architecture v1.0
Features	Drafting (RAG) · Precedent Search · Document Verification
Tech Stack	Next.js · Python/FastAPI · Neon PostgreSQL · Nginx · Docker
Auth	Clerk
Payments	Razorpay
Storage	Cloudflare R2
Phase 1	Drafting form→RAG→draft · Keyword search · Auth · Payments
Phase 2	Semantic search (pgvector) · Citation panel · HC ingestion · Draft history

Table of Contents

1	Full System Architecture	3
2	Drafting Feature — Data Flow	4
3	Precedent Search — Data Flow	5
4	Individual Service Architecture	6
4.1	Auth Service	6
4.2	Drafting Service	6
4.3	Search Service	6
4.4	Doc Verify Service	6
4.5	Payment Service	7
4.6	Ingestion Worker	7
5	Entity Relationship Diagram (ERD)	8
6	Database Schema — Full SQL	9
7	File Structure	11
8	Docker & Infrastructure	12
9	Phase Comparison	13
10	Key Technical Decisions	13

1. Full System Architecture

The platform is built as a **Modular Monolith** (Phase 1) evolving toward selective microservices (Phase 2+). All components run in Docker containers behind an Nginx reverse proxy. P2 additions (pgvector semantic search, HC ingestion) are shown with dashed outlines.

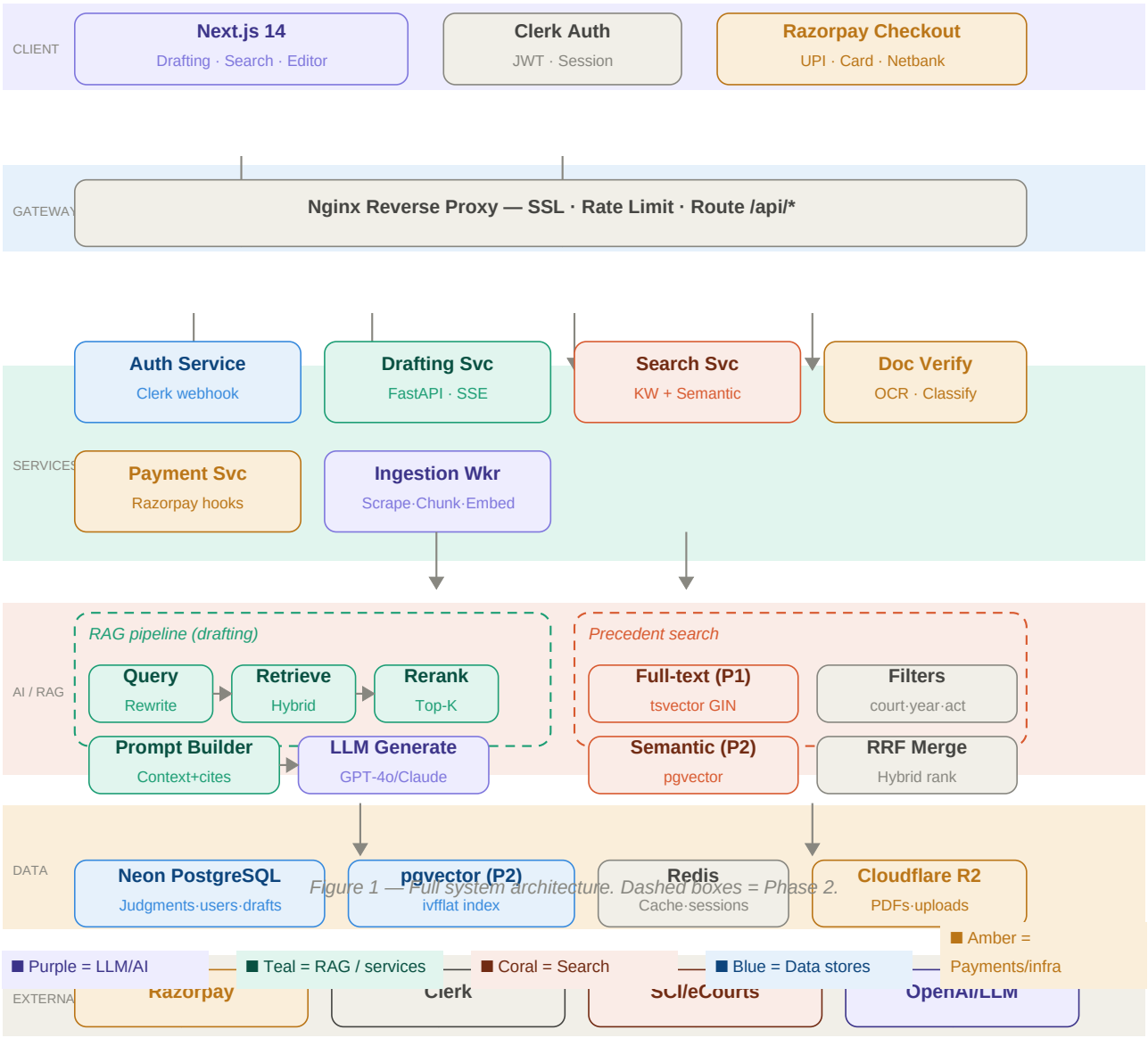


Figure 1 — Full system architecture. Dashed boxes = Phase 2.

2. Drafting Feature — Data Flow

Complete end-to-end flow from user filling the case form and uploading supporting documents through document verification, hybrid RAG retrieval, LLM generation, and real-time SSE streaming to an editable rich-text draft editor with autosave and export.

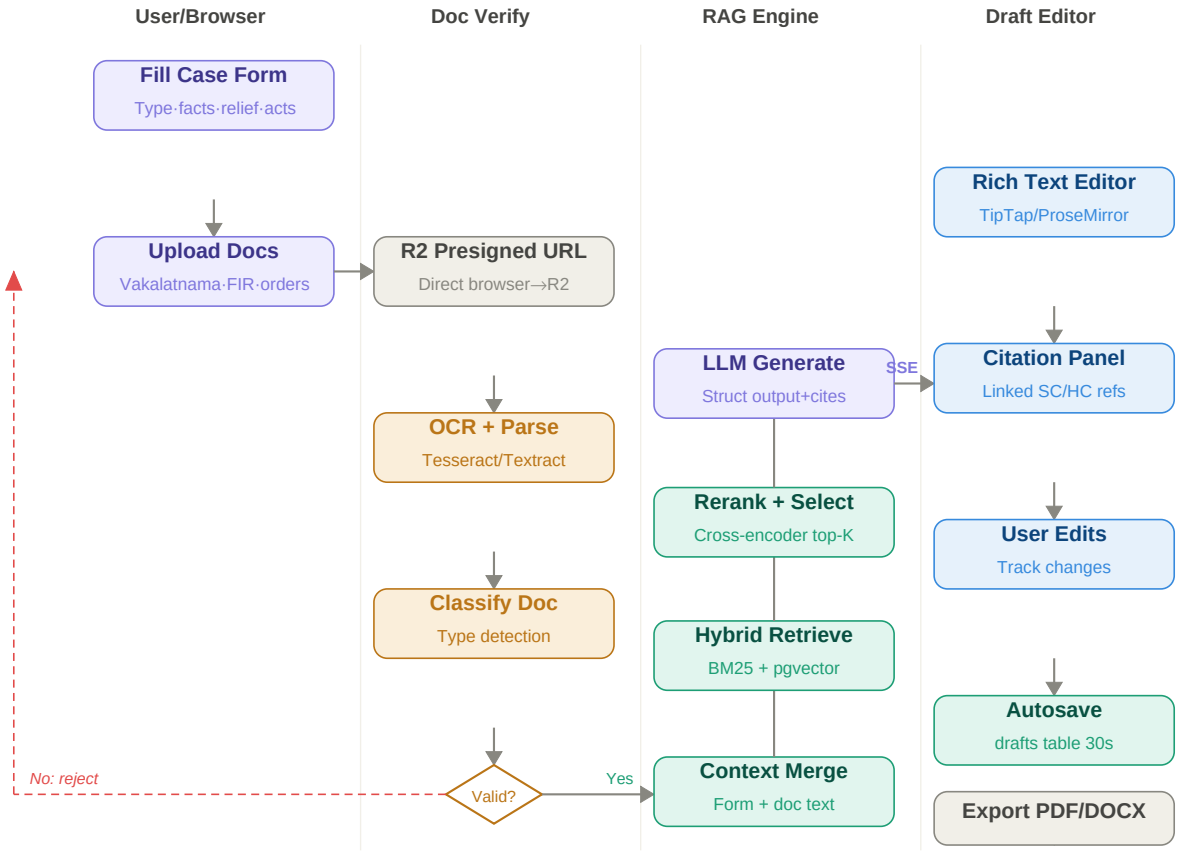


Figure 2 — Drafting data flow. Red dashed path = rejection on invalid document.

Document Verification Detail

When a user uploads a vakalatnama, FIR, or court order, the system: **(1)** generates a presigned Cloudflare R2 URL so the browser uploads directly (no server bandwidth cost), **(2)** runs OCR via **AWS Textract** (recommended for handwritten/scanned Indian court docs) or Tesseract, **(3)** classifies the document type using an LLM-based classifier, **(4)** rejects with a specific reason if mismatched or unreadable, **(5)** passes extracted text into the RAG context window.

The RAG retrieval step ("Hybrid Retrieve" in Figure 2) queries **two vectorized corpora at once**: the judgment/precedent corpus and the statutory code corpus (IPC/BNS, CrPC/BNSS, IEA/BSA, and other acts — see Section 3.1). This means a draft citing "culpable homicide" retrieves both the matching BNS/IPC section text and relevant Supreme Court judgments interpreting that section, in the same pass.

Editable Draft Editor

Uses **TipTap** (ProseMirror-based). The LLM streams output via **Server-Sent Events (SSE)** so the draft appears word-by-word. User edits freely after generation. Custom citation nodes link directly to the source judgment. Autosave fires every 30 seconds, writing draft_html to the drafts table. Export to PDF and DOCX is supported.

3. Precedent Search — Data Flow

Two parallel pipelines: **left** is the user-facing search (Phase 1: full-text BM25, Phase 2: semantic via pgvector). **Right** is the background ingestion worker that scrapes SCI and HC portals, chunks judgments, embeds them, and stores everything in Neon PostgreSQL.

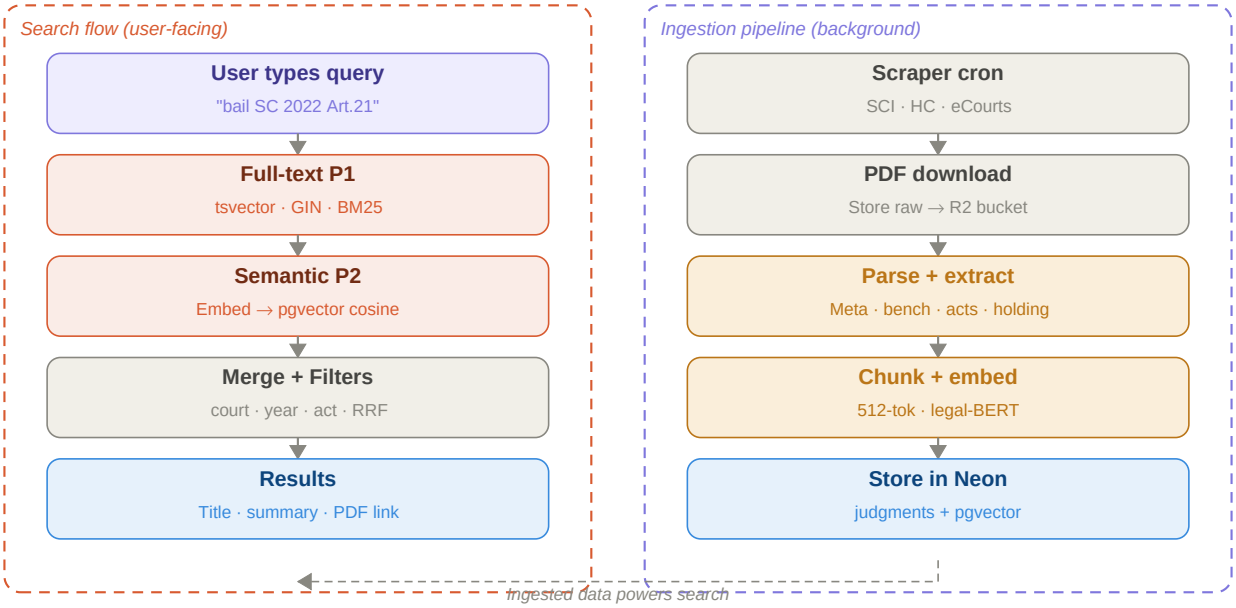


Figure 3 — Search (left) and background ingestion (right) pipelines.

Phase	Method	Index	Latency
P1 — Keyword	tsvector BM25	GIN index on search_vector	< 50ms
P2 — Semantic	pgvector cosine	ivfflat (lists=100)	< 200ms
P2 — Hybrid	RRF score merge	Both indexes	< 300ms

3.1 Legal Codes as a Second Knowledge Base

Beyond case judgments, the RAG pipeline retrieves from a separate, structurally distinct corpus: the **bare acts / statutory codes** — IPC, BNS, CrPC, BNSS, Evidence Act (IEA/BSA), CPC, Contract Act, IBC, POCSO, NDPS, and others. Both drafting and precedent search query this corpus alongside case law, since a citation like *"Section 302 IPC"* or *"Section 103 BNS"* needs to resolve to the exact statutory text — not just be inferred by the LLM.

Domain	Old Code	New Code (2023)	Chunking unit
Criminal offences	IPC, 1860	BNS, 2023	Section + illustration
Criminal procedure	CrPC, 1973	BNSS, 2023	Section + sub-clause
Evidence	Indian Evidence Act, 1872	BSA, 2023	Section
Civil procedure	CPC, 1908	— (unchanged)	Order + Rule
Contracts	Indian Contract Act, 1872	— (unchanged)	Section
Insolvency	IBC, 2016	— (unchanged)	Section + Schedule
Special laws	POCSO, UAPA, NDPS, PMLA, IT Act	— (unchanged)	Section

Why a separate table, not the judgments table: statutory text has a different update cadence (amendments are rare but must instantly invalidate old embeddings), a different structural granularity (section/sub-section vs. paragraph), and

needs a stable, versioned section-number lookup that judgments don't require. Keeping legal_codes and legal_code_sections separate from judgments and judgment_chunks avoids polluting one index with the other's ranking behavior, while both are queried together at retrieval time via a **fan-out search**.

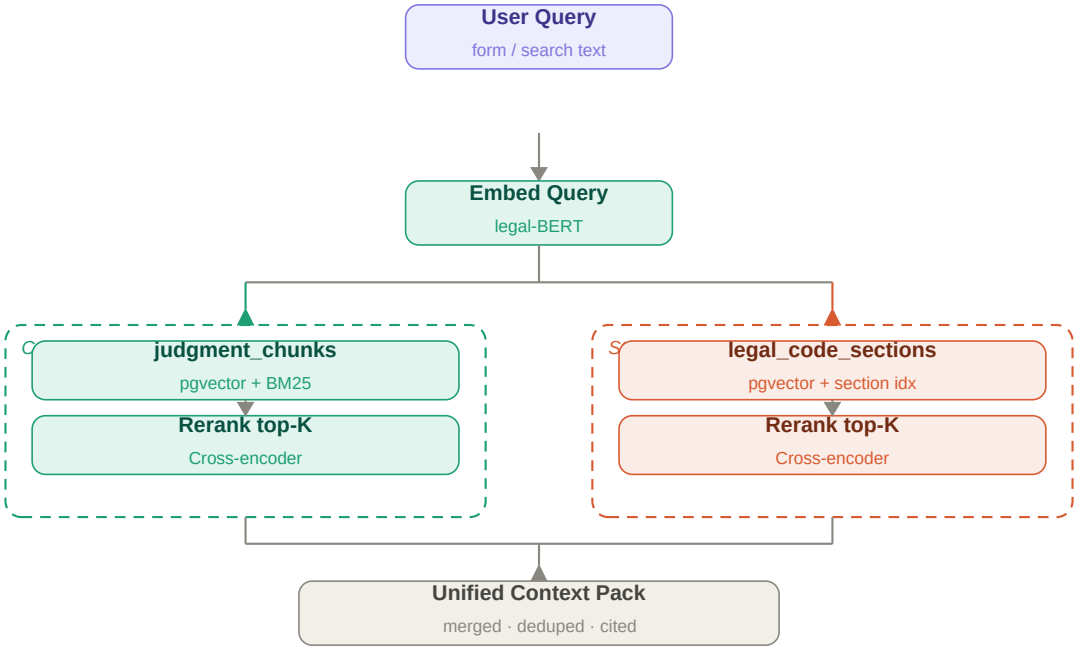
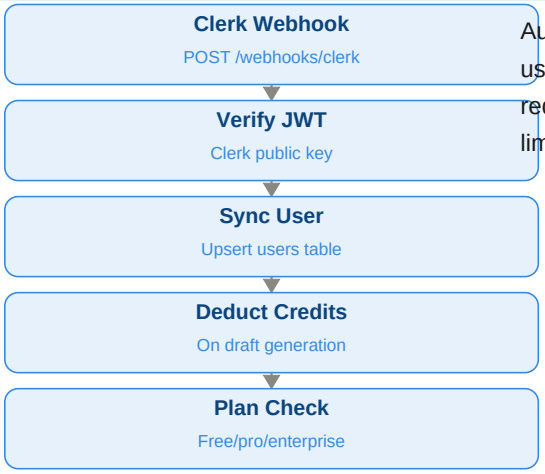
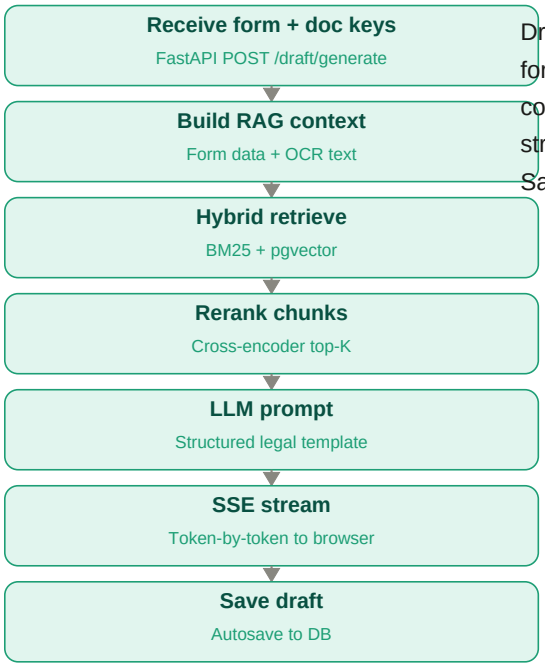


Figure 3.1 — Unified retrieval fans out across judgments + statutory codes, merges by relevance.

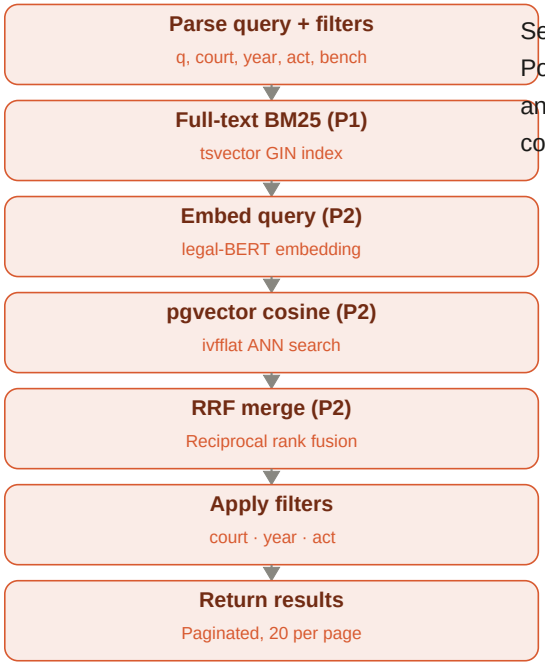
4. Individual Service Architecture



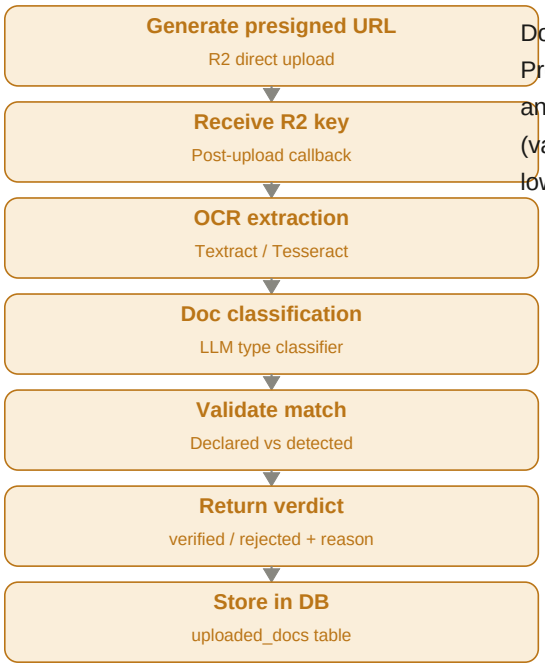
Auth Service handles Clerk webhook events (user.created, user.updated, subscription.changed), verifies JWT on every API request, syncs user records to Neon, and enforces plan-based credit limits before allowing draft generation.



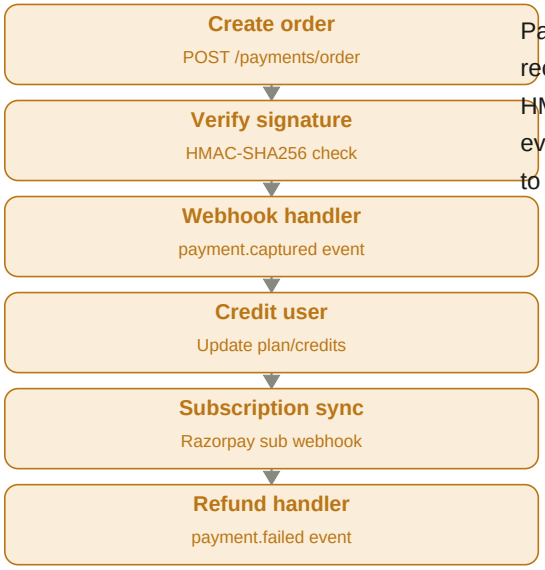
Drafting Service orchestrates the entire RAG pipeline. Accepts user form data and uploaded document R2 keys, builds a combined context, retrieves and reranks relevant judgment chunks, constructs a structured legal prompt, and streams the LLM response via SSE. Saves draft HTML and cited judgment IDs on completion.



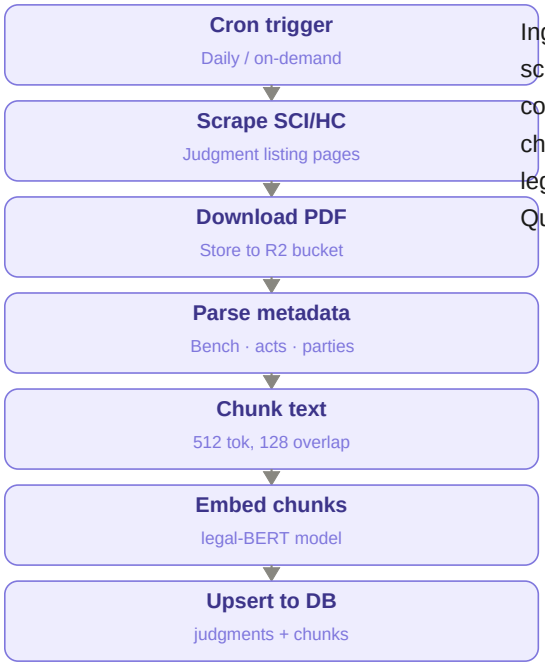
Search Service handles all precedent browsing. Phase 1 uses PostgreSQL full-text search. Phase 2 adds pgvector semantic search and merges results via Reciprocal Rank Fusion. Supports filter combinations: court, year, bench composition, acts cited.



Doc Verify Service manages secure document upload and validation. Presigned URLs allow direct browser-to-R2 upload. OCR extracts text and a lightweight LLM classifier identifies the document type (vakalatnama, FIR, bail order, charge sheet). Mismatch or low-confidence results in rejection with a user-visible reason.



Payment Service integrates Razorpay for one-time credit packs and recurring subscriptions. All webhook signatures are verified via HMAC-SHA256 before crediting any account. Subscription lifecycle events (created, charged, cancelled, expired) are handled and synced to the subscriptions table.



Ingestion Worker runs as a separate Docker container on a cron schedule. Scrapes SCI and HC portals, deduplicates via content_hash, downloads PDFs to R2, parses structured metadata, chunks the full text, generates embeddings using a fine-tuned legal-BERT model, and upserts everything to Neon PostgreSQL. Quality score is computed per judgment.

5. Entity Relationship Diagram

All database tables with columns, types, primary/foreign keys, and cardinality relationships. Tables are grouped by domain. The judgments table is your existing schema with two additions: embedding vector(1536) and search_vector tsvector.

users

Column	Type	Notes
id	UUID	PK
clerk_id	TEXT	UK, from Clerk
email	TEXT	
full_name	TEXT	
plan	TEXT	free|pro|enterprise
credits_remaining	INT	default 5
created_at	TIMESTAMPTZ	
updated_at	TIMESTAMPTZ	

drafts

Column	Type	Notes
id	BIGSERIAL	PK
user_id	UUID	FK → users
title	TEXT	
case_type	TEXT	Writ/Bail/Appeal
court	TEXT	SC/HC/District
form_data	JSONB	All user inputs
draft_html	TEXT	TipTap rich HTML
draft_text	TEXT	Plain text for search
citation_ids	BIGINT[]	FK → judgments[]
status	TEXT	draft|final|archived
created_at	TIMESTAMPTZ	
updated_at	TIMESTAMPTZ	

uploaded_docs

Column	Type	Notes
id	BIGSERIAL	PK
draft_id	BIGINT	FK → drafts
user_id	UUID	FK → users
original_filename	TEXT	
r2_key	TEXT	R2 object key
doc_type	TEXT	vakalatnama|FIR|order
ocr_text	TEXT	Extracted text
verify_status	TEXT	pending|ok|rejected
verify_reason	TEXT	Rejection message
uploaded_at	TIMESTAMPTZ	

judgments (existing + additions)

Column	Type	Notes
id	BIGSERIAL	PK
case_number	TEXT	Indexed
case_type	TEXT	Indexed
year	INTEGER	Indexed
judgment_date	TEXT	
bench	TEXT[]	
petitioner	TEXT	
respondent	TEXT	
acts_cited	TEXT[]	
cases_cited	TEXT[]	
full_text	TEXT	
content_hash	TEXT	UK, dedup
pdf_s3_key	TEXT	R2 path
summary	TEXT	
holding	TEXT	
embedding	VECTOR(1536)	NEW — P2
search_vector	TSVECTOR	NEW — P1 GIN

judgment_chunks

Column	Type	Notes
id	BIGSERIAL	PK
judgment_id	BIGINT	FK → judgments
chunk_index	INT	Order within doc
chunk_text	TEXT	512 token window
embedding	VECTOR(1536)	ivfflat indexed
page_number	INT	Source page

payments

Column	Type	Notes
id	BIGSERIAL	PK
user_id	UUID	FK → users
razorpay_order_id	TEXT	UK
razorpay_payment_id	TEXT	
amount_paise	INT	In paise (■×100)
currency	TEXT	default INR
status	TEXT	created|paid|failed
created_at	TIMESTAMPTZ	

legal_codes

Column	Type	Notes
id	BIGSERIAL	PK
code_name	TEXT	IPC · BNS · CrPC ...
short_code	TEXT	'IPC','BNS','CrPC'
year_enacted	INT	
status	TEXT	active|repealed
replaced_by_id	BIGINT	FK self, e.g. IPC →BNS

legal_code_sections

Column	Type	Notes
id	BIGSERIAL	PK
legal_code_id	BIGINT	FK → legal_codes
section_number	TEXT	'302', '103(1)'
title	TEXT	Section heading
section_text	TEXT	Full statutory text
embedding	VECTOR(1536)	ivfflat indexed
corresponds_to	TEXT	Old→new section map

subscriptions

Column	Type	Notes
id	BIGSERIAL	PK
user_id	UUID	FK → users
razorpay_sub_id	TEXT	UK
plan_name	TEXT	pro|enterprise
status	TEXT	active|cancelled
starts_at	TIMESTAMPTZ	
ends_at	TIMESTAMPTZ	

search_logs		
Column	**Type**	**Notes**
id	BIGSERIAL	PK
user_id	UUID	FK → users
query	TEXT	
search_type	TEXT	keyword|semantic
result_count	INT	
clicked_judgment_id	BIGINT	FK → judgments
searched_at	TIMESTAMPTZ	

Relationships

Parent				Child	Key
users	1	→	many	subscriptions	user_id FK
users	1	→	many	payments	user_id FK
users	1	→	many	drafts	user_id FK
users	1	→	many	uploaded_docs	user_id FK
users	1	→	many	search_logs	user_id FK
drafts	1	→	many	uploaded_docs	draft_id FK
judgments	1	→	many	judgment_chunks	judgment_id FK
legal_codes	1	→	many	legal_code_sections	legal_code_id FK
legal_codes	1	→	0/1	legal_codes	replaced_by_id FK (self, e.g. IPC→BNS)

6. Database Schema — Full SQL

Enable pgvector extension

```
-- Run once on Neon database
CREATE EXTENSION IF NOT EXISTS vector;
```

users table

```
CREATE TABLE users (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  clerk_id        TEXT UNIQUE NOT NULL,
  email           TEXT NOT NULL,
  full_name       TEXT,
  plan            TEXT DEFAULT 'free',   -- free | pro | enterprise
  credits_remaining INT  DEFAULT 5,
  created_at      TIMESTAMPTZ DEFAULT now(),
  updated_at      TIMESTAMPTZ DEFAULT now()
);
```

subscriptions table

```
CREATE TABLE subscriptions (
  id               BIGSERIAL PRIMARY KEY,
  user_id          UUID REFERENCES users(id) ON DELETE CASCADE,
  razorpay_sub_id  TEXT UNIQUE,
  plan_name        TEXT,
  status           TEXT,  -- active | cancelled | expired | paused
  starts_at        TIMESTAMPTZ,
  ends_at          TIMESTAMPTZ
);
```

payments table

```
CREATE TABLE payments (
  id                  BIGSERIAL PRIMARY KEY,
  user_id             UUID REFERENCES users(id),
  razorpay_order_id   TEXT UNIQUE,
  razorpay_payment_id TEXT,
  amount_paise        INT,
  currency            TEXT DEFAULT 'INR',
  status              TEXT,  -- created | paid | failed | refunded
  created_at          TIMESTAMPTZ DEFAULT now()
);
```

drafts table

```
CREATE TABLE drafts (
  id          BIGSERIAL PRIMARY KEY,
  user_id     UUID REFERENCES users(id) ON DELETE CASCADE,
  title       TEXT,
  case_type   TEXT,  -- Writ | Bail | Criminal Appeal | Civil Appeal
  court       TEXT,  -- Supreme Court | HC Delhi | HC Bombay ...
  form_data   JSONB,                  -- Full user form inputs
  draft_html  TEXT,                  -- TipTap rich text HTML
  draft_text  TEXT,                  -- Plain text for FTS on drafts
  citation_ids BIGINT[],             -- Judgment IDs cited in draft
  status      TEXT DEFAULT 'draft',  -- draft | finalized | archived
  created_at  TIMESTAMPTZ DEFAULT now(),
  updated_at  TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX idx_drafts_user ON drafts(user_id);
CREATE INDEX idx_drafts_status ON drafts(status);
```

uploaded_docs table

```
CREATE TABLE uploaded_docs (
  id                BIGSERIAL PRIMARY KEY,
  draft_id          BIGINT REFERENCES drafts(id) ON DELETE CASCADE,
  user_id           UUID   REFERENCES users(id),
  original_filename TEXT,
  r2_key            TEXT,       -- Cloudflare R2 object key
  doc_type          TEXT,       -- vakalatnama|fir|bail_order|charge_sheet
  ocr_text          TEXT,       -- Extracted via Textract/Tesseract
  verify_status     TEXT,       -- pending | verified | rejected
  verify_reason     TEXT,       -- Human-readable rejection reason
  uploaded_at       TIMESTAMPTZ DEFAULT now()
);
```

judgment_chunks table (Phase 2)

```
CREATE TABLE judgment_chunks (
  id           BIGSERIAL PRIMARY KEY,
  judgment_id  BIGINT REFERENCES judgments(id) ON DELETE CASCADE,
  chunk_index  INT,
  chunk_text   TEXT,
  embedding    VECTOR(1536),
  page_number  INT
);
CREATE INDEX ON judgment_chunks
  USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);
```

search_logs table

```
CREATE TABLE search_logs (
  id                BIGSERIAL PRIMARY KEY,
  user_id           UUID REFERENCES users(id),
  query             TEXT,
  search_type       TEXT,    -- keyword | semantic | hybrid
  result_count      INT,
  clicked_judgment_id BIGINT REFERENCES judgments(id),
  searched_at       TIMESTAMPTZ DEFAULT now()
);
```

legal_codes + legal_code_sections (statutory corpus)

```
-- Parent table: one row per Act/Code
CREATE TABLE legal_codes (
  id            BIGSERIAL PRIMARY KEY,
  code_name     TEXT NOT NULL,     -- 'Bharatiya Nyaya Sanhita, 2023'
  short_code    TEXT NOT NULL,     -- 'BNS', 'IPC', 'CrPC', 'BNSS'
  year_enacted  INT,
  status        TEXT DEFAULT 'active',  -- active | repealed
  replaced_by_id BIGINT REFERENCES legal_codes(id)  -- IPC row points to BNS row
);

-- Child table: one row per section, vectorized for RAG retrieval
CREATE TABLE legal_code_sections (
  id             BIGSERIAL PRIMARY KEY,
  legal_code_id  BIGINT REFERENCES legal_codes(id) ON DELETE CASCADE,
  section_number TEXT NOT NULL,      -- '302', '103(1)', '34'
  title          TEXT,               -- 'Punishment for murder'
  section_text   TEXT NOT NULL,      -- Full statutory text + illustrations
  embedding      VECTOR(1536),
  corresponds_to TEXT,               -- e.g. 'IPC S.302 -> BNS S.103'
  UNIQUE(legal_code_id, section_number)
```

```
);
CREATE INDEX ON legal_code_sections
  USING ivfflat (embedding vector_cosine_ops) WITH (lists = 50);
CREATE INDEX idx_lcs_section_num ON legal_code_sections(section_number);
```

Add to existing judgments table

```
-- Phase 1: full-text search column (generated, always updated)
ALTER TABLE judgments
  ADD COLUMN IF NOT EXISTS search_vector TSVECTOR
  GENERATED ALWAYS AS (
    to_tsvector('english',
      coalesce(title,'') || ' ' ||
      coalesce(summary,'') || ' ' ||
      coalesce(holding,'') || ' ' ||
      coalesce(full_text,''))
  ) STORED;
CREATE INDEX idx_judgments_fts ON judgments USING GIN(search_vector);

-- Phase 2: semantic embedding column
ALTER TABLE judgments
  ADD COLUMN IF NOT EXISTS embedding VECTOR(1536);
CREATE INDEX ON judgments
  USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);
```

7. File Structure

```
writuploaded/
■■■ frontend/                          # Next.js 14 App Router
■   ■■■ app/
■   ■   ■■■ (auth)/
■   ■   ■   ■■■ login/page.tsx
■   ■   ■■■ dashboard/
■   ■   ■   ■■■ draft/
■   ■   ■   ■   ■■■ page.tsx           # New draft form
■   ■   ■   ■   ■■■ [id]/page.tsx      # Saved draft view
■   ■   ■   ■   ■■■ [id]/edit/page.tsx # Edit existing draft
■   ■   ■   ■■■ search/
■   ■   ■   ■   ■■■ page.tsx           # Precedent browser
■   ■   ■   ■■■ account/page.tsx
■   ■   ■■■ api/                       # Thin Next.js API proxies
■   ■   ■   ■■■ upload-url/route.ts    # Get R2 presigned URL
■   ■   ■■■ layout.tsx
■   ■■■ components/
■   ■   ■■■ draft/
■   ■   ■   ■■■ DraftForm.tsx          # Case details input form
■   ■   ■   ■■■ DocUploader.tsx        # Document upload + verify
■   ■   ■   ■■■ DraftEditor.tsx        # TipTap rich text editor
■   ■   ■   ■■■ CitationPanel.tsx      # Referenced judgments sidebar
■   ■   ■   ■■■ ExportMenu.tsx         # PDF / DOCX export
■   ■   ■■■ search/
■   ■   ■   ■■■ SearchBar.tsx
■   ■   ■   ■■■ FilterPanel.tsx        # Court, year, act filters
■   ■   ■   ■■■ JudgmentCard.tsx       # Result card
■   ■   ■   ■■■ JudgmentModal.tsx      # Full text / summary view
■   ■   ■■■ shared/
■   ■       ■■■ Navbar.tsx
■   ■       ■■■ PlanBadge.tsx
■   ■■■ lib/
■   ■   ■■■ api.ts                     # Axios client + interceptors
■   ■   ■■■ clerk.ts                   # Auth helpers
■   ■   ■■■ r2.ts                      # R2 presigned URL request
■   ■■■ Dockerfile
■
■■■ backend/                           # Python FastAPI monolith
■   ■■■ app/
■   ■   ■■■ main.py                    # FastAPI entry, middleware
■   ■   ■■■ config.py                  # Pydantic-settings (env vars)
■   ■   ■■■ api/
■   ■   ■   ■■■ v1/
■   ■   ■       ■■■ drafting.py        # POST /draft/generate (SSE)
■   ■   ■       ■■■ search.py          # GET  /search/precedents
■   ■   ■       ■■■ judgments.py       # GET  /judgments/:id
■   ■   ■       ■■■ documents.py       # POST /documents/verify
■   ■   ■       ■■■ payments.py        # POST /payments/order
■   ■   ■       ■■■ webhooks.py        # POST /webhooks/razorpay
■   ■   ■       ■■■ auth.py            # POST /webhooks/clerk
■   ■   ■■■ core/
■   ■   ■   ■■■ security.py            # Clerk JWT verification
■   ■   ■   ■■■ database.py            # SQLAlchemy async + asyncpg
■   ■   ■   ■■■ redis.py               # Redis client + cache helpers
■   ■   ■   ■■■ r2.py                  # Cloudflare R2 client
■   ■   ■■■ modules/
■   ■   ■   ■■■ drafting/
■   ■   ■   ■   ■■■ service.py         # RAG orchestration
■   ■   ■   ■   ■■■ prompt_builder.py  # Legal prompt templates
■   ■   ■   ■   ■■■ post_processor.py  # Citation validation
■   ■   ■   ■■■ rag/
■   ■   ■   ■   ■■■ retriever.py       # Hybrid BM25 + pgvector
■   ■   ■   ■   ■■■ reranker.py        # Cross-encoder reranking
■   ■   ■   ■   ■■■ embedder.py        # legal-BERT embeddings
■   ■   ■   ■   ■■■ query_rewriter.py  # LLM query expansion
■   ■   ■   ■■■ search/
```

```
■   ■   ■   ■   ■■■ service.py         # Search orchestration
■   ■   ■   ■   ■■■ filters.py         # Court/year/act filter builder
■   ■   ■   ■■■ documents/
■   ■   ■   ■   ■■■ ocr.py             # Textract / Tesseract wrapper
■   ■   ■   ■   ■■■ classifier.py      # Doc type LLM classifier
■   ■   ■   ■   ■■■ service.py         # Verify orchestration
■   ■   ■   ■■■ payments/
■   ■   ■   ■   ■■■ razorpay.py        # Razorpay client wrapper
■   ■   ■   ■   ■■■ service.py         # Order + webhook logic
■   ■   ■   ■■■ scraper/
■   ■   ■       ■■■ sci_scraper.py     # Supreme Court scraper
■   ■   ■       ■■■ hc_scraper.py      # High Court scrapers
■   ■   ■       ■■■ codes_ingester.py  # IPC/BNS/CrPC/BNSS bare-act ingestion
■   ■   ■       ■■■ chunker.py         # Text chunking (512 tok)
■   ■   ■       ■■■ pipeline.py        # End-to-end ingestion
■   ■   ■■■ models/
■   ■       ■■■ judgment.py           # SQLAlchemy ORM model
■   ■       ■■■ legal_code.py         # legal_codes + sections ORM
■   ■       ■■■ user.py
■   ■       ■■■ draft.py
■   ■       ■■■ payment.py
■   ■       ■■■ uploaded_doc.py
■   ■■■ alembic/                      # DB migrations
■   ■   ■■■ env.py
■   ■   ■■■ versions/
■   ■■■ tests/
■   ■   ■■■ test_drafting.py
■   ■   ■■■ test_search.py
■   ■   ■■■ test_payments.py
■   ■■■ Dockerfile
■
■■■ infra/
■   ■■■ docker-compose.yml            # Local development
■   ■■■ docker-compose.prod.yml       # Production
■   ■■■ nginx/
■       ■■■ nginx.conf                # Proxy + SSL config
■
■■■ .env.example                      # All required env vars
```

8. Docker & Infrastructure

docker-compose.prod.yml (outline)

```
version: "3.9"
services:
  nginx:
    image: nginx:alpine
    ports: ["80:80", "443:443"]
    volumes: ["./infra/nginx/nginx.conf:/etc/nginx/conf.d/default.conf"]
    depends_on: [frontend, backend]

  frontend:
    build: ./frontend
    environment:
      - NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY
      - NEXT_PUBLIC_API_URL=/api

  backend:
    build: ./backend
    environment:
      - DATABASE_URL          # Neon connection string (pooled)
      - REDIS_URL
      - CLERK_SECRET_KEY
      - CLERK_WEBHOOK_SECRET
      - RAZORPAY_KEY_ID
      - RAZORPAY_KEY_SECRET
      - R2_ACCOUNT_ID
      - R2_ACCESS_KEY_ID
      - R2_SECRET_ACCESS_KEY
      - R2_BUCKET_NAME
      - OPENAI_API_KEY        # or ANTHROPIC_API_KEY
      - AWS_TEXTRACT_KEY      # for doc OCR
    depends_on: [redis]

  redis:
    image: redis:7-alpine
    volumes: [redis-data:/data]
    command: redis-server --save 60 1

  scraper:
    build: ./backend
    command: python -m app.modules.scraper.pipeline
    profiles: ["scraper"]        # Only runs when explicitly invoked
    environment:
      - DATABASE_URL
      - R2_BUCKET_NAME

volumes:
  redis-data:
```

Environment Variables (.env.example)

Variable	Service	Description
DATABASE_URL	backend	Neon PostgreSQL connection string (pooled)
REDIS_URL	backend	Redis connection string
CLERK_SECRET_KEY	backend	Clerk secret for JWT verification
CLERK_WEBHOOK_SECRET	backend	For verifying Clerk webhook signatures
RAZORPAY_KEY_ID	backend/frontend	Razorpay key ID
RAZORPAY_KEY_SECRET	backend	Razorpay key secret for HMAC verify
R2_ACCOUNT_ID	backend/scraper	Cloudflare account ID
R2_ACCESS_KEY_ID	backend/scraper	R2 access key

`R2_SECRET_ACCESS_KEY`	backend/scraper	R2 secret key
`R2_BUCKET_NAME`	backend/scraper	R2 bucket name
`OPENAI_API_KEY`	backend	For LLM generation and query rewriting
`AWS_ACCESS_KEY_ID`	backend	For AWS Textract OCR
`AWS_SECRET_ACCESS_KEY`	backend	For AWS Textract OCR
`NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY`	frontend	Clerk public key for client auth

9. Phase Comparison

Capability	Phase 1 (MVP)	Phase 2
Search	tsvector full-text BM25	+ pgvector semantic + hybrid RRF merge
Embeddings	None	legal-BERT-base-uncased or multilingual-e5
Citation panel	Text list of references	Clickable — opens judgment inline
Draft history	Last saved version only	Full version history with diff view
Courts ingested	Supreme Court of India	Allahabad, Bombay, Delhi, Madras HCs
Doc upload OCR	Tesseract (free)	AWS Textract (handwritten/scanned)
Doc types	PDF only	PDF + image scans
Analytics	Basic search logs	Search analytics dashboard
Auth	Clerk JWT	Clerk + SSO (Google/Microsoft)
Architecture	Modular Monolith	Extract ingestion as separate service

10. Key Technical Decisions

Modular Monolith over Microservices

For a 1–5 person team at MVP stage, a modular monolith is the right choice. Each feature lives in its own Python module/package with clear interfaces. The ingestion worker is the first candidate to extract when it needs independent GPU scaling. Microservices add operational overhead (distributed tracing, inter-service auth, separate CI) that provides no business value at this stage.

Embedding Model: legal-BERT over OpenAI

Use sentence-transformers/legal-bert-base-uncased or InLegalBERT for embeddings. These models are fine-tuned on Indian and Commonwealth legal corpora, giving significantly better semantic similarity for legal text than general-purpose OpenAI embeddings. They are also free to run and eliminate per-token embedding costs at scale.

Chunking Strategy

Split judgments at paragraph boundaries, targeting 512 tokens with 128-token overlap. Store chunk_index and page_number per chunk so citations link back to the exact page. Use the judgment heading (parties + year) as prefix on every chunk to preserve context.

TipTap for the Draft Editor

TipTap (ProseMirror-based) supports custom node types (citation links that open judgment modals), track changes extension, collaborative editing for future law firm accounts, and clean HTML/DOCX/Markdown export. It streams well with SSE — each token appends to the editor.

SSE over WebSockets for Streaming

Server-Sent Events are sufficient for unidirectional draft streaming. SSE works through Nginx without special configuration, reconnects automatically on disconnect, and is natively supported in all modern browsers without a library.

Razorpay Webhook Security

Always verify the X-Razorpay-Signature header using HMAC-SHA256 with your webhook secret before crediting any account. Never trust the payment amount from the webhook payload — look it up from the order you created. Store idempotency keys to prevent double-crediting.

Neon PostgreSQL Connection Pooling

Neon's free tier limits concurrent connections. Use asyncpg with a pool of max 10 connections and route read-heavy search queries through Neon's built-in read replica. Use PgBouncer in transaction mode if connection pressure increases.